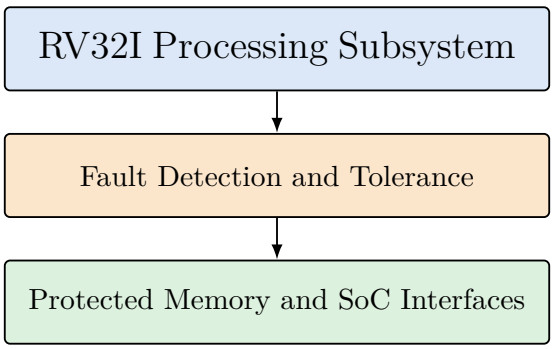
Fault-Tolerant RISC-V RV32I System-on-Chip for CubeSats

Radiation-Induced Fault Detection,
Error Correction and Recovery



Technical Research Report

Focus: RV32I Base Integer Instruction Set Only

GitHub Repository: `https://github.com/sujithvarmap/riscv-base`

Reference Video: `https://youtu.be/ga93fR30WEw`

Contents

1 Introduction **1**
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Project Objectives . . . 1
- 1.4 Project Scope . . . 2

2 RISC-V and the RV32I Base ISA **3**
- 2.1 What Is RISC-V? . . . 3
- 2.2 RV32I Overview . . . 3
- 2.3 Architectural Registers . . . 3
- 2.4 RV32I Instruction Categories . . . 4
- 2.5 RV32I Instruction Formats . . . 4
  - 2.5.1 R-Type . . . 5
  - 2.5.2 I-Type . . . 5
  - 2.5.3 S-Type . . . 5
  - 2.5.4 B-Type . . . 5
  - 2.5.5 U-Type . . . 5
  - 2.5.6 J-Type . . . 5
  - 2.5.7 Immediate Reconstruction . . . 6

3 Processor Microarchitecture **7**
- 3.1 Five-Stage Pipeline . . . 7
- 3.2 Datapath Components . . . 7
- 3.3 Hazards and Forwarding . . . 8
- 3.4 Memory-Mapped Peripherals . . . 8

4 Radiation Effects in Digital Systems **9**
- 4.1 Radiation Environment . . . 9
- 4.2 Single-Event Upset . . . 9
- 4.3 Other Radiation Effects . . . 10
- 4.4 Fault Propagation . . . 10

5 Triple Modular Redundancy **11**
- 5.1 Principle . . . 11
- 5.2 TMR Architecture . . . 11
- 5.3 Example . . . 11
- 5.4 TMR in a Processor . . . 12
- 5.5 TMR Limitations . . . 12
- 5.6 Hardware Overhead . . . 12

6 Error-Correcting Codes **13**
6.1 Motivation . . . 13
6.2 Data Efficiency . . . 13
6.3 Hamming Codes . . . 13
6.4 Memory Scrubbing . . . 14

7 Proposed Fault-Tolerant SoC Architecture **15**
7.1 Architecture Overview . . . 15
7.2 Proposed SoC Diagram . . . 15
7.3 Fault Monitor . . . 16
7.4 Recovery Strategy . . . 16

8 Existing Repository and Development Plan **17**
8.1 Existing RV32I Project . . . 17
8.2 Existing Features Versus Proposed Features . . . 17
8.3 Development Phases . . . 17

9 Fault-Injection Verification **19**
9.1 Purpose . . . 19
9.2 Fault-Injection Targets . . . 19
9.3 Suggested Test Plan . . . 20
9.4 Evaluation Metrics . . . 20

10 Architecture Comparison **21**

11 Limitations and Future Work **22**
11.1 Limitations . . . 22
11.2 Future Work . . . 22

12 Conclusion **23**

A Example RV32I Assembly **24**

B Resources and References **25**

List of Figures

2.1 RV32I instruction formats. The B-type and J-type immediate fields are distributed across noncontiguous instruction bits and must be reconstructed by the decoder. . . 5

3.1 Conventional five-stage processor pipeline. 7
3.2 Simplified processor datapath. This functional view omits detailed pipeline registers, multiplexers, forwarding paths, and branch-control wiring. 8

4.1 Illustration of a single-bit upset. 9

5.1 Conceptual Triple Modular Redundancy architecture. 11

6.1 Simplified ECC memory protection and scrubbing path. 14

7.1 Proposed fault-tolerant RV32I SoC. The bus, memory, diagnostic, and recovery connections must match the final RTL implementation. 15

List of Tables

2.2 Principal RV32I instruction categories 4

4.1 Selected radiation-related effects 10

8.1 Implementation status and proposed extensions 17

9.1 Example fault-injection scenarios 20

10.1 Proposed design variants 21

1 Introduction

1.1 Background

Small satellites, including CubeSats, depend on electronic systems that must operate reliably under challenging environmental conditions. One important challenge is exposure to ionizing radiation.

High-energy particles can deposit charge in semiconductor devices and disturb the behavior of digital circuits. Depending on the affected location, a radiation-induced event may change a stored bit, produce a transient pulse, corrupt a processor register, or cause an incorrect control decision.

A processor used in a spacecraft must therefore be evaluated not only for its computational capability, but also for its ability to detect errors, prevent incorrect outputs, and recover from faults.

RISC-V provides an open instruction set architecture that can serve as the foundation for a custom processor. This project focuses exclusively on the RV32I base integer instruction set and investigates how additional hardware protection could improve the reliability of an RV32I-based SoC.

1.2 Problem Statement

A digital processor stores and manipulates binary information. A single-bit upset may change a stored value from zero to one or from one to zero.

If the affected bit belongs to an instruction, register, program counter, memory word, or control signal, the processor may perform an unintended operation.

For example, an incorrect address may direct a memory access to the wrong location, while an incorrect branch condition may change the execution path.

The central problem is to design and evaluate a small RV32I-based SoC that can detect, mask, correct, or recover from selected faults while considering hardware area, power, timing, and performance.

1.3 Project Objectives

The main objectives are:

1. Understand the RV32I base integer ISA.
2. Study the datapath and pipeline of an RV32I processor.
3. Identify vulnerable processor and memory state.
4. Investigate Triple Modular Redundancy (TMR).
5. Investigate Error-Correcting Codes (ECC).
6. Develop a fault-injection verification strategy.
7. Evaluate area, timing, power estimates, and fault coverage.
8. Investigate fault monitoring and recovery.

1.4 Project Scope

The processor ISA considered in this report is RV32I. The report does not introduce the M, A, F, D, C, or other optional instruction-set extensions.

TMR, ECC, watchdogs, and recovery mechanisms are additional hardware design techniques. They are not provided automatically by the RV32I instruction set.

The report distinguishes between the existing educational processor implementation in the project repository and the proposed fault-tolerant extensions.

The proposed architecture is a research design. It is not claimed to be radiation-qualified or ready for flight.

2 RISC-V and the RV32I Base ISA

2.1 What Is RISC-V?

RISC-V is an open instruction set architecture (ISA). An ISA specifies the instructions supported by a processor and the architectural behavior visible to software.

An ISA is different from a microarchitecture. The ISA describes what instructions do, while the microarchitecture describes how a processor executes them internally.

For example, two processors can implement RV32I while using different pipeline depths, execution units, branch strategies, and memory interfaces.

2.2 RV32I Overview

RV32I is the 32-bit base integer instruction set. Its principal characteristics include:

- Thirty-two 32-bit architectural integer registers, named `x0` through `x31`.
- A program counter (PC).
- Integer arithmetic and logical operations.
- Load and store operations.
- Conditional branches and jump instructions.
- Fixed 32-bit instruction encodings in the base ISA.
- A load-store programming model.

The standard RV32I base ISA has no compressed 16-bit instruction encoding. The compressed extension is separate and is not included in this project scope.

2.3 Architectural Registers

Register	ABI name	Purpose
`x0`	zero	Always reads as zero. Writes are ignored.
`x1`	ra	Return address under the standard ABI.
`x2`	sp	Stack pointer.
`x3`	gp	Global pointer.
`x4`	tp	Thread pointer.
`x5-x7`	t0–t2	Temporary registers.
`x8-x9`	s0–s1	Saved registers; `x8` may also be called `fp`.
`x10-x17`	a0–a7	Function arguments and return values under the standard ABI.
`x18-x27`	s2–s11	Saved registers.

Register	ABI name	Purpose
x28–x31	t3–t6	Temporary registers.

The ABI names represent common software conventions. The architectural register identifiers are x0 through x31.

2.4 RV32I Instruction Categories

Table 2.2: Principal RV32I instruction categories

Category	Examples	Purpose
Arithmetic	ADD, SUB, ADDI	Integer addition and subtraction.
Logical	AND, OR, XOR, ANDI	Bitwise logic and masking.
Shift	SLL, SRL, SRA	Logical and arithmetic shifts.
Comparison	SLT, SLTU, SLTI	Signed and unsigned comparisons.
Loads	LB, LH, LW, LBU, LHU	Read data from memory.
Stores	SB, SH, SW	Write data to memory.
Branches	BEQ, BNE, BLT, BGE	Conditional control flow.
Jumps	JAL, JALR	Unconditional control transfer and link generation.
Upper immediates	LUI, AUIPC	Construct constants and PC-relative addresses.
System/control	ECALL, EBREAK, FENCE	Environment calls, breakpoints, and ordering-related operations.

The complete implementation must follow the official RISC-V Unprivileged ISA Specification.

2.5 RV32I Instruction Formats

Every base RV32I instruction is 32 bits long. The principal formats are R, I, S, B, U, and J.

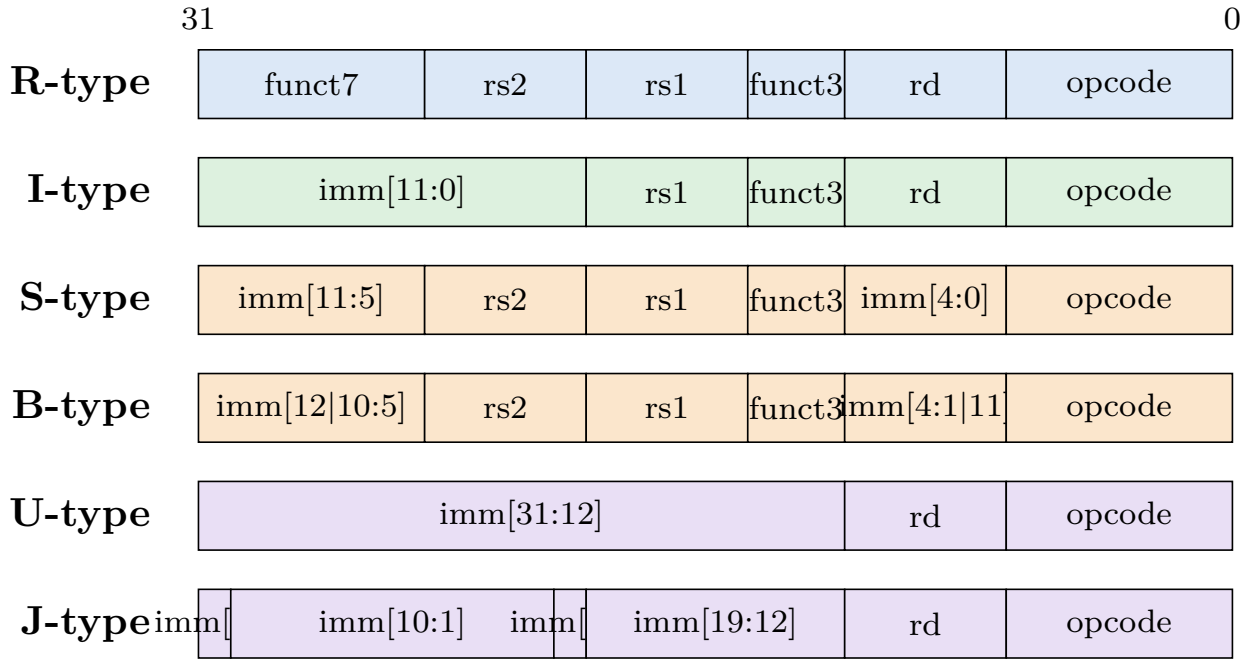


Figure 2.1: RV32I instruction formats. The B-type and J-type immediate fields are distributed across noncontiguous instruction bits and must be reconstructed by the decoder.

2.5.1 R-Type

R-type instructions normally use two source registers and one destination register. They are used for register-register arithmetic, logical operations, and shifts.

2.5.2 I-Type

I-type instructions include arithmetic operations with immediate operands and loads. The immediate field is sign-extended where required by the instruction.

2.5.3 S-Type

S-type instructions encode stores. The immediate is split between bits 31–25 and bits 11–7.

2.5.4 B-Type

B-type instructions encode conditional branches. The branch immediate is reconstructed from the instruction fields and is sign-extended.

The effective branch target is calculated relative to the current program counter.

2.5.5 U-Type

U-type instructions contain a 20-bit upper immediate field and a destination register. They support operations such as `LUI` and `AUIPC`.

2.5.6 J-Type

J-type instructions encode a jump immediate distributed across multiple fields. `JAL` uses the resulting offset to calculate a PC-relative target and can save a return address.

2.5.7 Immediate Reconstruction

The decoder must not treat every immediate as a contiguous field. In particular, B-type and J-type immediates require bit rearrangement and an implicit zero in the least significant position.

Incorrect immediate reconstruction can produce incorrect branch targets and jump addresses.

3 Processor Microarchitecture

3.1 Five-Stage Pipeline

A five-stage pipeline is a common implementation choice for RV32I, but it is not required by the ISA.

1. **IF – Instruction Fetch:** Fetch the instruction using the program counter.
2. **ID – Instruction Decode:** Decode the instruction and read source registers.
3. **EX – Execute:** Perform ALU operations, comparisons, and effective-address calculations.
4. **MEM – Memory Access:** Perform data memory operations for loads and stores.
5. **WB – Writeback:** Write the result into the destination register when required.

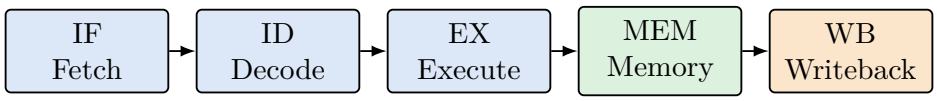


Figure 3.1: Conventional five-stage processor pipeline.

Pipeline registers separate stages and retain intermediate values between clock cycles.

3.2 Datapath Components

A basic RV32I processor may contain:

- Program counter.
- Instruction interface.
- Instruction decoder and control unit.
- Integer register file.
- Immediate generator.
- Arithmetic logic unit.
- Branch comparison and target-generation logic.
- Data memory interface.
- Pipeline registers.
- Forwarding and hazard-control logic.
- Result-selection and register-writeback logic.

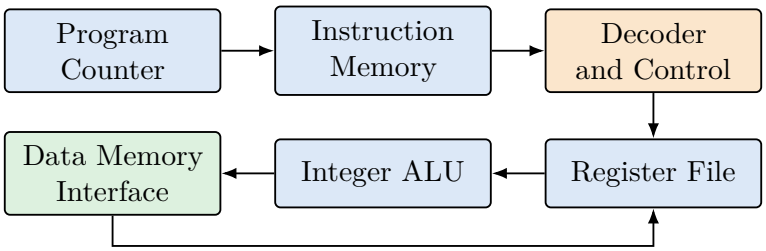


Figure 3.2: Simplified processor datapath. This functional view omits detailed pipeline registers, multiplexers, forwarding paths, and branch-control wiring.

3.3 Hazards and Forwarding

A pipelined processor may encounter data hazards when an instruction depends on the result of an earlier instruction.

Forwarding can provide results directly from later pipeline stages. A hazard unit may stall the pipeline when forwarding cannot resolve a dependency.

Control hazards can occur when branches or jumps change the instruction-fetch path. The processor must handle these cases according to its pipeline design.

These mechanisms are important to verify before adding fault-tolerance hardware.

3.4 Memory-Mapped Peripherals

A complete SoC may connect the processor to peripherals through memory-mapped registers.

Possible peripherals include:

- Timer and watchdog.
- UART for debugging and telemetry.
- SPI or I2C controller.
- GPIO for digital inputs and outputs.
- Fault-status and diagnostic registers.

These are SoC design choices and do not require new RV32I instructions.

4 Radiation Effects in Digital Systems

4.1 Radiation Environment

Spacecraft electronics may encounter energetic protons, heavy ions, and secondary particles generated through interactions with spacecraft materials.

The actual radiation risk depends on the orbit, mission duration, shielding, semiconductor technology, device design, and operating conditions.

A radiation-tolerant architecture must therefore be evaluated against a defined mission environment.

4.2 Single-Event Upset

A Single-Event Upset (SEU) is a change in a stored logic state caused by a particle interaction.

For example, an eight-bit value may change from

$$D_{\text{original}} = 10110010$$

to

$$D_{\text{corrupted}} = 10100010.$$

One bit has changed from one to zero.

If the affected value is used by the processor, the error may influence an arithmetic operation, memory address, branch decision, or control signal.

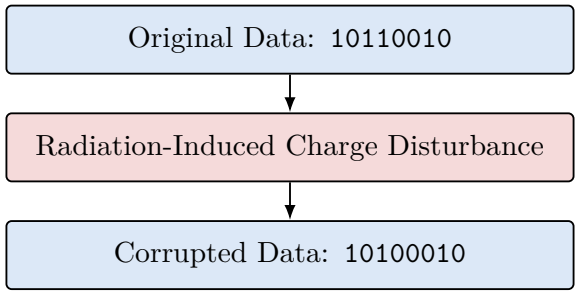


Figure 4.1: Illustration of a single-bit upset.

4.3 Other Radiation Effects

Table 4.1: Selected radiation-related effects

Effect	Description	Potential consequence
SEU	A stored logic state changes	Incorrect register or memory value.
SET	A transient pulse occurs in logic	An incorrect value may be sampled.
Multiple-bit upset	Multiple bits are affected	Some ECC codes may be unable to correct the error.
SEL	A parasitic current path becomes conductive	Excess current and possible device damage.
Total ionizing dose	Accumulated radiation exposure	Long-term changes in device characteristics.

These effects require different mitigation techniques. For example, ECC can help with certain memory errors, while current limiting may be necessary to mitigate latch-up damage.

4.4 Fault Propagation

Not every upset causes a system-level failure. A corrupted temporary value may be overwritten before it is used.

The consequence depends on the affected location, execution timing, and subsequent operations. Four important concepts are:

- **Detection:** Identify that an error occurred.
- **Correction:** Restore a supported correct value.
- **Masking:** Prevent a faulty value from affecting the accepted result.
- **Recovery:** Restore operation after a fault.

A complete design should specify which fault classes it detects, corrects, masks, or leaves unhandled.

5 Triple Modular Redundancy

5.1 Principle

Triple Modular Redundancy (TMR) replicates a logic function three times and uses a majority voter.

For Boolean outputs, the majority function is

$$Y = (A \wedge B) \vee (A \wedge C) \vee (B \wedge C).$$

If two replicas agree on the correct output and the third replica differs, the majority voter can mask the faulty output.

5.2 TMR Architecture

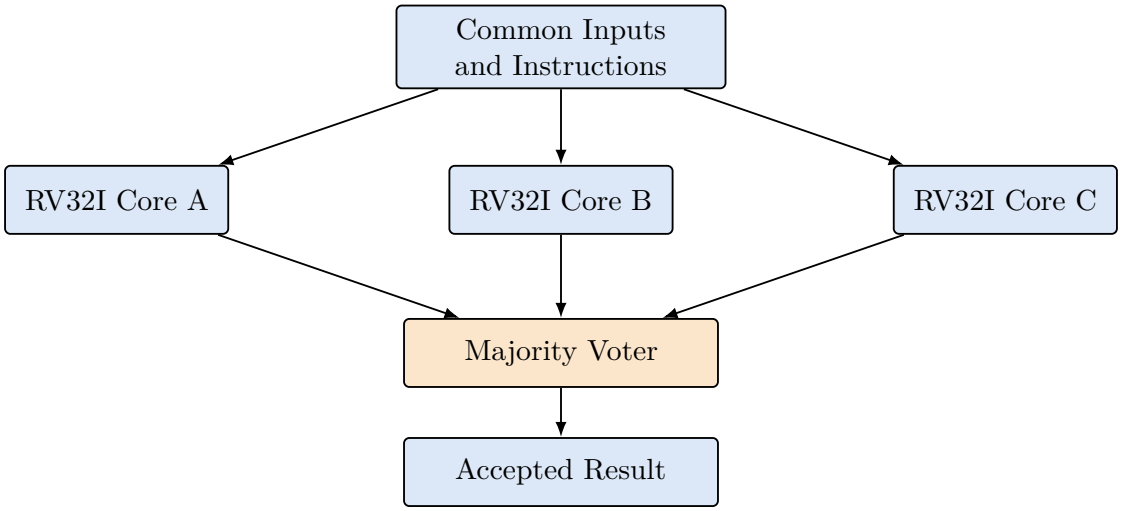


Figure 5.1: Conceptual Triple Modular Redundancy architecture.

5.3 Example

Suppose three replicas calculate the same operation:

$$A = 17, \qquad B = 21, \qquad C = 17.$$

The majority result is

$$Y = \text{majority}(17, 21, 17) = 17.$$

The mismatching result can also generate a fault indication.

5.4 TMR in a Processor

For a processor, comparing only the final ALU result is not enough to protect the entire system.

A practical TMR implementation must consider:

- Program counters.
- Pipeline registers.
- Register-file state.
- Branch decisions.
- Memory write operations.
- Interrupts and exception state.
- Shared bus and peripheral transactions.

The replicas must remain synchronized and receive consistent inputs.

5.5 TMR Limitations

TMR does not guarantee protection against every fault. Its effectiveness depends on the voting boundary, replica independence, and the protection of shared resources.

Potential failure sources include:

- Multiple replicas failing together.
- A faulty majority voter.
- A corrupted shared input or memory.
- Synchronization errors.
- Faults in shared interconnect or reset logic.

The voter and critical shared resources must also be considered in the fault-tolerance strategy.

5.6 Hardware Overhead

Let A_C represent the area of one processor core, A_V the voter area, and A_S the shared-resource area. An approximate TMR area is

$$A_{\mathrm{TMR}} \approx 3A_C + A_V + A_S.$$

The total SoC area is not necessarily exactly three times the baseline area because some infrastructure may be shared, while other protection circuitry adds further overhead.

If each stored data word is triplicated, the unique data fraction is

$$\eta_{\mathrm{TMR}} = \frac{1}{3} \times 100\% = 33.33\%.$$

This is a raw data-storage payload fraction, not a universal measure of processor efficiency.

6 Error-Correcting Codes

6.1 Motivation

Error-Correcting Codes (ECC) add check bits to data. An encoder generates the check information, while a decoder checks the received codeword.

Depending on the code, the decoder may detect errors, correct supported error patterns, or report an uncorrectable error.

ECC is especially useful for instruction memory, data memory, and other stored state.

It does not automatically protect unreplicated combinational logic or guarantee correct execution.

6.2 Data Efficiency

Suppose a memory code stores k data bits and r check bits. Its total codeword length is

$$n = k + r.$$

The data payload efficiency is

$$\eta_{\mathrm{ECC}} = \frac{k}{k+r} \times 100\%.$$

For an illustrative code with 64 data bits and 6 check bits,

$$\eta_{\mathrm{ECC}} = \frac{64}{64+6} \times 100\% \approx 91.43\%.$$

This illustrates how a code can provide check information with less storage overhead than storing three copies of every data bit.

The actual overhead and correction capability depend on the chosen code and memory organization.

6.3 Hamming Codes

For a Hamming code with k data bits and r parity bits, the usual minimum parity-bit condition for single-error correction is

$$2^r \geq k + r + 1.$$

A suitable Hamming code can correct a single-bit error in its codeword.

Additional parity structures can support single-error correction and double-error detection. The exact guarantees depend on the code construction.

6.4 Memory Scrubbing

Memory scrubbing periodically reads protected memory, checks the stored data, corrects supported errors, and writes corrected values back.

Scrubbing can reduce the time that recoverable errors remain stored. Its effectiveness depends on the scrub interval, memory size, radiation environment, and probability of additional errors before correction.

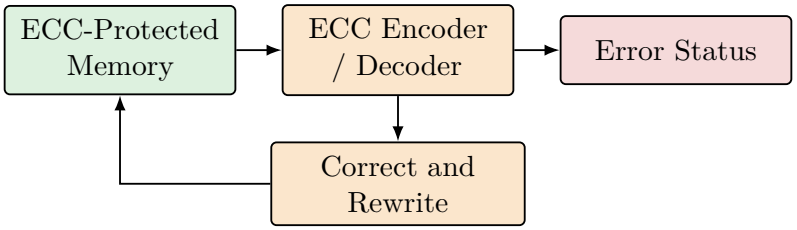


Figure 6.1: Simplified ECC memory protection and scrubbing path.

7 Proposed Fault-Tolerant SoC Architecture

7.1 Architecture Overview

The proposed system uses an RV32I processing subsystem connected to memory and peripherals. Additional protection circuitry is proposed around the processing and memory paths. The design may include:

- RV32I processing subsystem.
- Bus interconnect.
- ECC-protected instruction and data memory.
- Fault monitor and diagnostic registers.
- Watchdog timer.
- TMR voter in a replicated implementation.
- Recovery controller.
- Satellite communication and I/O interfaces.

7.2 Proposed SoC Diagram

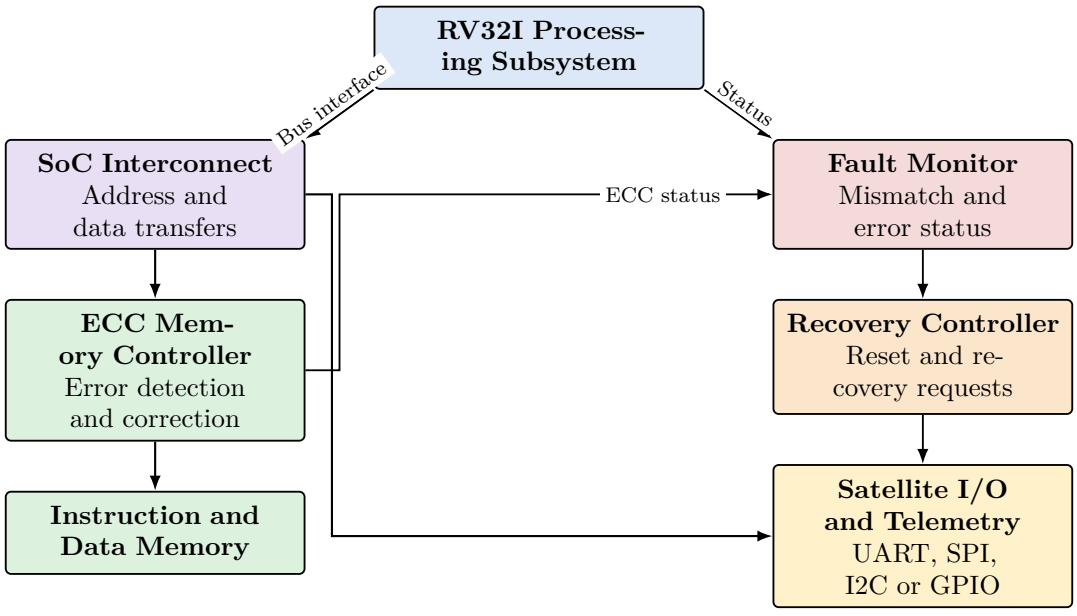


Figure 7.1: Proposed fault-tolerant RV32I SoC. The bus, memory, diagnostic, and recovery connections must match the final RTL implementation.

The diagram is a conceptual architecture rather than a claim that all blocks have already been implemented.

In particular, the ECC controller, TMR logic, watchdog, and recovery controller must be designed and verified before they can be treated as implemented features.

7.3 Fault Monitor

The fault monitor records and reports errors detected by the processing subsystem, voter, ECC controller, and other protected blocks.

Possible diagnostic registers include:

- Fault status.
- Fault source identifier.
- Error counter.
- Last reported fault address.
- Watchdog status.
- Recovery status.

The register map must define which events are latched, how status is cleared, and how software acknowledges errors.

The fault monitor itself is not inherently immune to radiation. Critical state and reset paths require appropriate protection.

7.4 Recovery Strategy

A recovery procedure may use the following steps:

1. Detect an error or mismatch.
2. Prevent an unverified result from causing an unsafe external side effect.
3. Record fault information when the logging path is trustworthy.
4. Correct data when a supported correction mechanism exists.
5. Reset or reinitialize the affected subsystem when appropriate.
6. Verify that operation has resumed correctly.

A reset alone does not necessarily correct corrupted memory. Persistent errors may require scrubbing, reloading a trusted firmware image, or isolating a failed component.

8 Existing Repository and Development Plan

8.1 Existing RV32I Project

The current processor project is available at:

`https://github.com/sujithvarmap/riscv-base`

The repository is the starting point for the processor implementation and verification work.

The project structure includes educational RV32I-style processor components such as the ALU, control unit, register file, forwarding logic, hazard logic, and pipeline registers.

The repository README describes a factorial demonstration that calculates $5! = 120$.

The exact implementation should be checked against the official RV32I specification and tested using an appropriate instruction-level verification suite.

8.2 Existing Features Versus Proposed Features

Table 8.1: Implementation status and proposed extensions

Feature	**Status**	**Purpose**
RV32I-style processor datapath	Existing project	Base for the research implementation.
ALU and register file	Existing project	Integer execution and architectural state.
Forwarding and hazard logic	Existing project	Manage pipeline dependencies.
TMR processor replicas	Proposed	Mask selected faults affecting one replica.
ECC memory controller	Proposed	Detect and correct supported memory errors.
Fault monitor	Proposed	Record error indications and diagnostic information.
Watchdog and recovery	Proposed	Detect stalled operation and initiate recovery.
Radiation qualification	Not established	Requires additional analysis and testing.

The table separates the current processor work from features that remain to be implemented. A feature should be moved to the implemented category only after the corresponding RTL and verification results are available.

8.3 Development Phases

1. **Baseline verification:** Verify RV32I instruction behavior and pipeline operation.

2. **Memory subsystem:** Define the memory interface and implement a suitable ECC scheme.
3. **Fault injection:** Introduce controlled faults in registers, memory, and control signals.
4. **TMR implementation:** Replicate the selected logic and implement majority voting.
5. **Recovery logic:** Integrate watchdog, diagnostic registers, and recovery behavior.
6. **Synthesis:** Compare baseline and protected designs under consistent constraints.
7. **Validation:** Run regression tests and document remaining fault classes and limitations.

9 Fault-Injection Verification

9.1 Purpose

Fault injection deliberately corrupts selected bits or signals in simulation to evaluate how the design responds.

It is useful for measuring logical fault coverage, but it does not reproduce every physical radiation effect. Real susceptibility also depends on process technology, layout, operating conditions, and the radiation environment.

9.2 Fault-Injection Targets

Possible targets include:

- General-purpose registers.
- Program counter.
- Pipeline registers.
- ALU outputs.
- Branch and control signals.
- Instruction memory.
- Data memory.
- ECC check bits.
- Majority-voter outputs.

Each test should define the fault location, injection time, expected behavior, and method of checking the result.

9.3 Suggested Test Plan

Table 9.1: Example fault-injection scenarios

Test	Injected fault	Expected observation
T1	One protected memory bit flips	ECC corrects the error if it is within the code's correction capability.
T2	An uncorrectable memory pattern is injected	An error is reported and unsafe data is not silently accepted.
T3	One TMR core produces a different result	The majority result is accepted when the other two agree and the voter is functioning correctly.
T4	Multiple TMR replicas disagree	The system reports a mismatch; correctness cannot be assumed without further recovery logic.
T5	A watchdog timeout occurs	The configured recovery procedure is initiated.
T6	Program counter corruption is injected	The implemented state-protection mechanism is tested.
T7	Voter output is corrupted	The voter-protection strategy is evaluated.

9.4 Evaluation Metrics

The proposed designs can be evaluated using:

- Logic and cell area.
- Maximum operating frequency.
- Estimated power under comparable workloads.
- Fault detection coverage.
- Fault correction and masking rates.
- Unhandled fault rate.
- Recovery latency.
- Memory overhead due to ECC.

If N eligible injected faults are tested and N_d are detected, empirical detection coverage is

$$C_{\text{det}} = \frac{N_d}{N} \times 100\%.$$

The denominator must be defined carefully. Detection, correction, masking, and safe recovery are different outcomes and should be reported separately.

10 Architecture Comparison

Table 10.1: Proposed design variants

Variant	**Protection**	**Evaluation goal**
A	Baseline RV32I	Measure baseline area, timing, power, and fault behavior.
B	RV32I with ECC	Evaluate memory-error correction and reporting.
C	TMR RV32I	Evaluate masking of faults affecting one replica.
D	TMR and ECC	Evaluate combined protection and associated overhead.

All variants should use consistent workloads, clock constraints, memory assumptions, synthesis settings, and fault-injection models.

The main research question is whether selective redundancy and ECC can reduce hardware overhead while preserving a defined level of fault coverage.

11 Limitations and Future Work

11.1 Limitations

The proposed architecture does not guarantee survival under every radiation condition.

TMR may fail under multiple or common-mode faults. ECC has finite correction capability. Shared buses, voters, reset logic, and unprotected control state can remain single points of failure.

RTL simulation and synthesis estimates do not replace radiation testing or component qualification.

11.2 Future Work

Future work may investigate:

- Selective redundancy instead of replicating the entire processor.
- Protection of program-counter and pipeline state.
- ECC selection based on memory width and error assumptions.
- Voter protection and replica synchronization.
- Fault-aware firmware and safe recovery procedures.
- FPGA-based fault injection.
- Technology-specific radiation characterization.

The goal is to find a practical balance between fault coverage, area, power, performance, and recovery complexity.

12 Conclusion

This report presents a proposed fault-tolerant SoC architecture based on the RV32I base integer instruction set for CubeSat applications.

RV32I provides the processor's instruction-level foundation, while additional hardware provides fault tolerance. TMR can mask selected faults affecting one replica when the other two agree. ECC can correct supported memory errors. Fault monitoring and recovery logic can help manage errors that are detected but not directly corrected.

The recommended development path begins with a verified RV32I processor, adds memory protection and fault injection, and then compares TMR with selective redundancy.

A meaningful research result should quantify area, timing, power, and fault coverage while documenting assumptions and remaining single points of failure.

A design should not be described as space-qualified without the relevant radiation analysis, validation, and qualification evidence.

A Example RV32I Assembly

The following sequence illustrates integer addition using standard RV32I instructions.

```
    .text
    .globl _start

_start:
    addi x5, x0, 12
    addi x6, x0, 5
    add  x7, x5, x6

    # x7 contains 17 after execution.
```

This example illustrates instruction behavior. It is not a complete firmware image or a complete CubeSat startup routine.

B Resources and References

[R1] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture.*

`https://docs.riscv.org/reference/isa/`

[R2] RISC-V International, *RISC-V Specifications.*

`https://riscv.org/technical/specifications/`

[R3] Project repository, *RISC-V Base Processor.*

`https://github.com/sujithvarmap/riscv-base`

[R4] Video reference supplied for this project.

`https://youtu.be/ga93fR30WEw`

[R5] NASA Small Spacecraft Systems Virtual Institute.

`https://www.nasa.gov/smallsat-institute/`

[R6] NASA Electronic Parts and Packaging Program.

`https://nepp.nasa.gov/`

[R7] European Space Agency.

`https://www.esa.int/`